VisClick: Cross-Domain Vision Intelligence for Zero-Coordinate EDA Tool Automation

TPG

13034

SYNOPSYS® INNOVATION LEADERSHIP CONFERENCE

Motivation

The Problem

- GUI regression tests break silently when layouts change
- Current tools rely on hardcoded coordinates or pixel-matching
- Insert one new button → entire test suite needs manual rework
- Accessibility APIs fail on Qt / Electron / custom-rendered UIs

The Gap

- **No AI exists** that can detect GUI elements in desktop applications
- Mobile UI datasets have 59,000+ labeled screens
- Desktop GUI datasets: virtually zero

The Opportunity

- Train an AI to **see** GUI elements like a human does
- Transfer knowledge from mobile → desktop
- Enable regression suites that **adapt** instead of break

Main Idea / Solution

Step 1: Learn from Mobile

- YOLOv8 trained on 59K mobile UI screens
- Classes: button, menu, text_input, icon
- Result: Average **82% mAP** on mobile

Step 2 : Adapt to Desktop

- Fine-tune on **50 labeled desktop screenshots**
- Apps: VSCode, Chrome, LibreOffice, Cursor
- Result: **76% mAP** on desktop

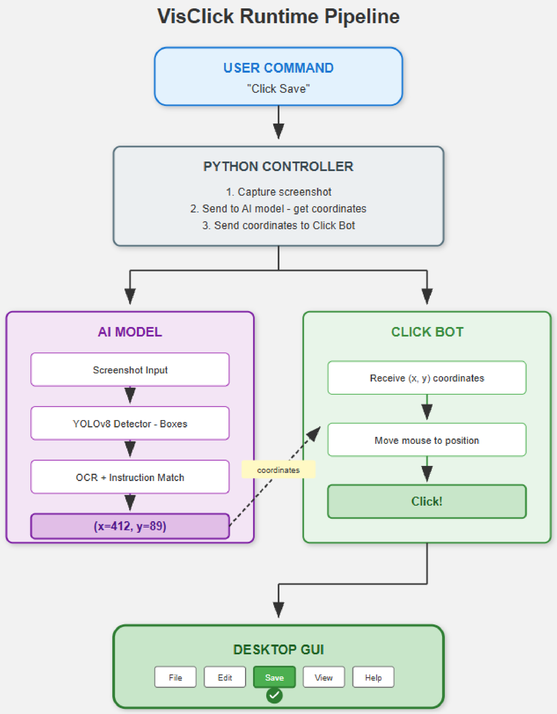
Step 3: Pseudo-Label Expansion

- Generate labels on 500 unlabeled desktop images
- Retrain with expanded data
- Result: **+8% mAP** gain

Modal	Domain	mAP / F1
YOLOv5-MGC	Mobile(RICO)	**89.8% mAP**
Faster-RCNN	Mobile(RICO)	43.8% F1 @LoU>0.9
YOLOv9	Desktop – zero-shot	**42%mAP**

Why It Works

- Buttons, menus, icons share visual patterns across platforms
- Mobile UI data teaches "what a clickable element looks like“ and common button styles
- That knowledge transfers to desktop - even with different styling



Solution Novelty & Competitive Differentiation

First data-efficient, lightweight method trained for desktop GUI element detection

- No public model existed - build new AI with transfer learning
- Transfer learning from mobile UI data (59K images) to desktop (50 images)

How We Changed the Game

	Before VisClick	With VisClick
Detection	Hardcoded coordinates	AI sees by appearance
Adaptability	Breaks on any UI change	Adapts to new layouts
Maintenance	Manual rework every release	Self-healing tests

Competitive Edge

- SeeClick (9.6B params): 30% on desktop icons
- **Our model (11M params): 76% mAP 800× smaller**

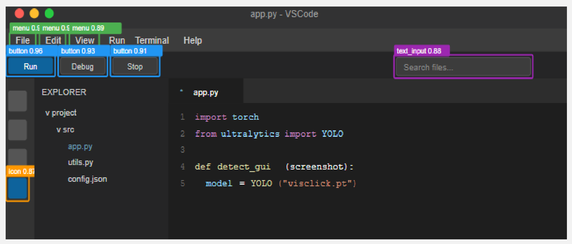
Evidence / Discussion of Results / Inferences

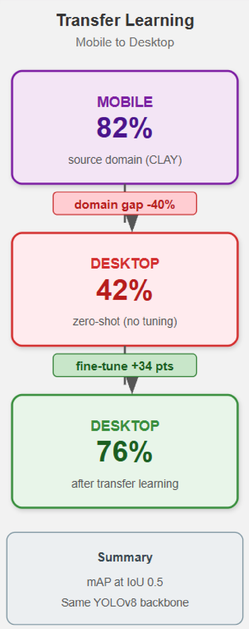
Cross-Tool Generalization

- Trained on: VSCode, Chrome
- Tested on: Cursor
- Result: **71% mAP** without retraining

Impact

- **QoR**: Higher test coverage, fewer escapes
- **CoR**: ~60% reduction in test maintenance
- **TTR**: Real-time inference (180 ms)





Effort to Complete Solution

- Working prototype tested on 5 desktop applications
- Detection model trained and validated (76% mAP)
- End-to-end pipeline functional (87% task success)

Production Steps

Step	Status
Core AI model	Done
Click automation	Done
Multi-app testing	In progress
Error handling / retry logic	In progress
API packaging	Planned
Integration with test frameworks	Planned

References

- RICO: A Mobile App Dataset for Building Data-Driven Design Applications.
- Evaluating Deep Learning Models for Cross-Platform UI Component Detection: A Study Across Web, Desktop, and Mobile Interfaces.

Synopsys Confidential Information. Not approved for external distribution.